Software and Data Engineering

Course Project Report

A Multi-Agent RAG Framework for Regulatory Compliance Checking of Software Requirements

Understanding, implementation and evaluation of Das et al., *ACM Transactions on Software Engineering and Methodology*, 2026 (doi:10.1145/3785472)

Team members
Tushar Kant (B23ES1033)
Pragay (B23ES1024)

Implementation repository: github.com/pack0shades/SDE_project (commit a8487da)

07 November 2026

Abstract

Regulations such as the EU General Data Protection Regulation (GDPR) and the EU Data Act place obligations on software systems that are easiest and cheapest to meet when they are caught at the requirements stage. Checking a Software Requirements Specification (SRS) against a long legal text by hand is slow and error-prone. This report describes our study and implementation of the multi-agent Retrieval-Augmented Generation (RAG) framework proposed by Das et al. In this framework, two compliance-checker agents built on language models from different vendors independently assess an SRS against a regulation, a report-analyzer agent resolves their disagreements through a structured revision dialogue, and a human reviewer accepts, rejects or escalates every finding, with decisions stored in a feedback memory that influences later runs.

We built the complete framework as a small Python package called *regcheck* (about 1,700 lines), together with a command-line interface, a Streamlit review application and an evaluation suite that reproduces scaled-down versions of the paper's Tables 4 to 7. All language-model access uses free tiers from three vendors (Mistral, Google Gemini and Cohere). On GDPR, the framework flagged 23 issues in the E-Store SRS against 6 for a single-prompt baseline, and agreement between the two checkers rose from 17 to 21 common findings after one refinement round. RAGAs evaluation gave context recall between 0.7 and 1.0 and showed that the advanced retrieval pipeline helps on long regulations but not on short SRS documents. The report sets out the paper, our design, every deliberate departure from the paper, the engineering problems we met, the results, and an honest account of merits, limitations and future work.

Contents

1. Introduction **4**

1.1 Problem statement 4

1.2 Objectives 4

1.3 Scope of this report 4

2. Background: the reference paper **4**

2.1 Overview 4

2.2 Research questions 4

2.3 The four components 5

2.4 Explainability 5

2.5 Hyperparameters 5

2.6 The paper's evaluation 6

3. System architecture of our implementation **6**

3.1 End-to-end flow 6

3.2 Repository layout 7

3.3 Data model 7

4. Detailed design of each component **8**

4.1 LLM access layer (llm.py) 8

4.2 Document Pre-processor (ingest.py) . . . 8
4.3 Vector store and retriever (store.py) . . . 9
4.4 Chain-of-thought question answering (rag.py) . . . 9
4.5 Compliance Checker agent (checker.py) . . . 9
4.6 Report Analyzer agent (analyzer.py) . . . 10
4.7 Agent communication loop (pipeline.py) . . . 10
4.8 Human-in-the-loop review and memory (app.py, memory.py) . . . 11
4.9 Explainability . . . 11
4.10 Prompt design . . . 11
5. Technology stack and configuration 11
6. Design decisions and departures from the paper 12
6.1 Guiding principles . . . 12
6.2 Departures from the paper . . . 13
6.3 Other notable decisions . . . 13
7. Implementation process and practical lessons 13
7.1 Phased plan . . . 13
7.2 Problems met and how they were solved . . . 14
8. Testing 14
9. Evaluation 15
9.1 Setup . . . 15
9.2 RAG quality (Table 4) . . . 15
9.3 Baseline vs advanced RAG (Table 5) . . . 16
9.4 Case studies (Table 6) . . . 16
9.5 Framework vs monolithic baseline (Table 7) . . . 17
10. Merits 17
11. Limitations and demerits 18
12. What we learnt 18
13. Future work 19
13.1 Immediate next steps . . . 19
13.2 Further improvements . . . 19
14. Conclusion 19
15. References 20
16. Appendix: running the system 20

1. Introduction

1.1 Problem statement

Software that handles personal or connected-product data must comply with regulations whose text runs to dozens or hundreds of articles. Non-compliance discovered late, during testing, audit or after release, is expensive to fix and can lead to legal penalties. The requirements phase is the natural place to detect such problems, but requirements engineers are rarely legal experts and regulations are long, cross-referenced and full of exceptions.

The project goal, as set for the course, is to develop a multi-agent RAG framework that automatically analyzes software requirements and identifies potential violations of relevant regulatory policies. Multiple AI agents collaborate using large language models (LLMs) and RAG to retrieve regulatory information, detect non-compliant requirements and suggest appropriate revisions, with human oversight over the final decisions.

1.2 Objectives

- Study the paper by Das et al. in depth and understand each of its components and evaluation methods.
- Implement the full framework: document pre-processing, retrieval, two compliance-checker agents, a report-analyzer agent, the agent-communication loop, human-in-the-loop review with feedback memory, and explainability features.
- Reproduce the paper's evaluation at a scale feasible for a student team using free-tier LLM access.
- Record every deliberate departure from the paper, with its reason, and assess the merits and limitations of the result.

1.3 Scope of this report

Section 2 summarises the paper. Sections 3 and 4 describe our system architecture and each component in detail. Section 5 lists the technology stack and configuration. Section 6 records the design decisions and departures from the paper. Section 7 describes how the work was carried out and the practical problems we solved. Sections 8 and 9 cover testing and evaluation. Sections 10 to 12 discuss merits, limitations and lessons learnt, and Section 13 sets out future work, including the additional SRS documents and QA tests planned next.

2. Background: the reference paper

2.1 Overview

Das, Deb, Chaki and Cortesi propose a framework that combines LLMs, prompt engineering and RAG to partially automate the verification of software requirements against regulations. The central idea is multi-agent communication: distinct agents independently evaluate requirements and then reconcile their findings, so that the final report does not depend on a single model's biases. A human-in-the-loop stage keeps a person responsible for every decision.

2.2 Research questions

RQ	Question	Paper's answer in brief
RQ1	Can LLMs check compliance while taking the software's operating context into account?	Yes, through chain-of-thought prompting and targeted retrieval.
RQ2	Can the framework be transparent and reliable?	Through multi-agent cross-validation, explainability (audit trails) and human review.

RQ	Question	Paper's answer in brief
RQ3	How should the efficiency of the approach be evaluated?	RAGAs metrics for the RAG component, plus case studies and crowdsourced feedback.
RQ4	Can the approach adapt to other regulations and evolving standards?	Partly, through a model-agnostic design and memory updates.

Table 1. Research questions of the reference paper.

2.3 The four components

A. Document Pre-processor

Regulation and SRS documents are split by an LLM agent into semantically self-contained chunks. The chunking agent splits nested conditions into separate propositions, resolves pronouns into explicit noun phrases, keeps exception clauses together with the rule they qualify and preserves cross-references. Chunks are embedded with *bge-large-en-v1.5* and stored in FAISS, with separate stores for policies and requirements.

B. Compliance Checker agents

Two agents, CC_Agent1 (Mixtral 8x7B) and CC_Agent2 (Command R+), each check the SRS independently. The prompting combines a persona ("As a GDPR compliance expert..."), chain-of-thought (CoT) query decomposition into three or four sub-questions, and contextual scaffolding that restricts the model to the retrieved context. The task is split into four sub-prompts: (1) list the requirements relevant to the regulation, (2) map each to the relevant articles, (3) rate each as compliant, partially compliant or non-compliant, and (4) identify gaps and suggest an improved requirement. Retrieval uses a KNN retriever with parent-document retrieval and Maximal Marginal Relevance (MMR), followed by a cross-encoder re-ranker (*mxbai-rerank-large-v1*) to counter the "lost in the middle" problem. Prompts were refined through expert error analysis; for example, a missed password-storage issue led to a sub-prompt that requires encryption at rest and in transit under GDPR Article 32.

C. Report Analyzer agent

The RA_Agent (Claude 3 Opus) uses six prompts: (i) compare the two reports, (ii) find the issues one agent missed, (iii) validate the rationale of the flagging agent, (iv) write a re-evaluation message to the agent concerned, (v) review the updated reports and (vi) compile the final report with a confidence score per issue. It can optionally consult web search, using a query abstraction protocol that replaces project-specific details with generic legal categories so that no proprietary content leaves the organisation.

D. Human-in-the-loop

Experts see an issue summary, the regulation mapping, a visualisation of each agent's rationale and the confidence score, and choose Accept, Reject or Escalate with a justification. Short-term memory uses RedisGraph with an LRU cache and long-term memory uses Neo4j with vector retrieval, storing expert feedback that refines later prompts.

2.4 Explainability

The paper presents three explainability features: a retrieval-path explanation (document, chunk and supporting sentence identifiers), step-by-step rationale generation tied to the cited evidence, and a transparent view of the dynamically generated prompts.

2.5 Hyperparameters

Parameter	Value in the paper
Embedding model	BAAI/bge-large-en-v1.5
Vector index	FAISS IndexFlatL2
Maximum chunk size	2,000 tokens
Top-k retrieval	15
Re-ranker	mixedbread-ai/mxbai-rerank-large-v1
CC agent temperature	0.2
RA agent	Claude 3 Opus, temperature 0.1

Table 2. Hyperparameters reported in the paper (its Table 2).

2.6 The paper's evaluation

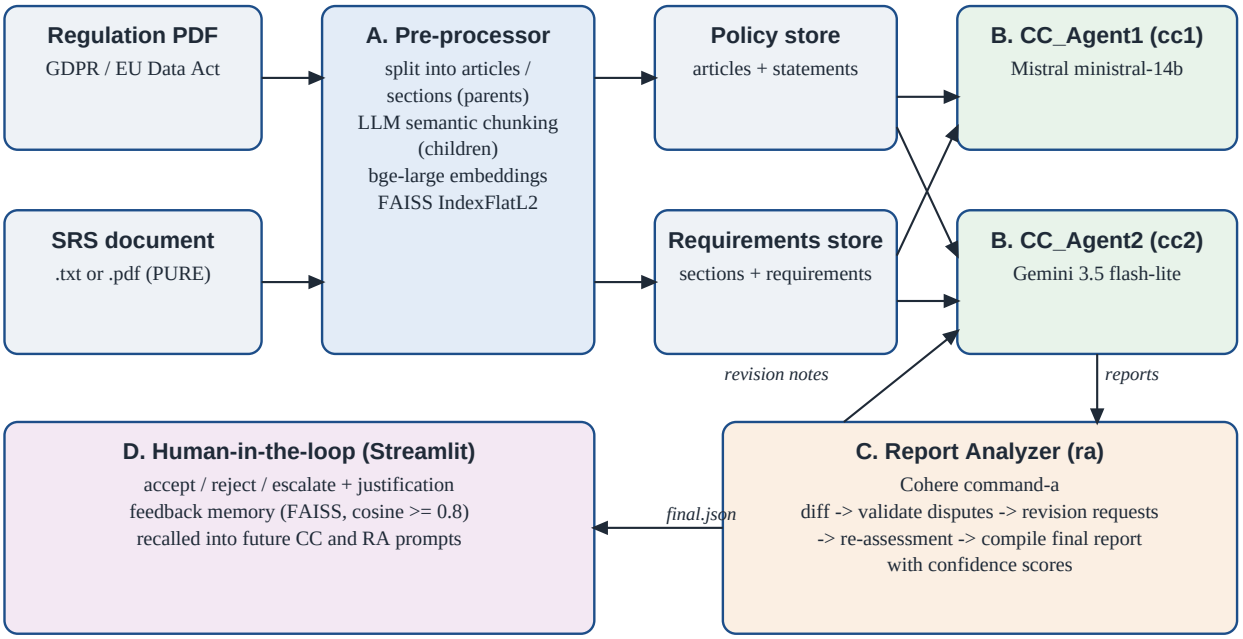
- **RAG quality (Tables 4 and 5):** RAGAs context precision, context recall, answer relevancy and faithfulness on eight document sets (GDPR, Data Act and six PURE SRS documents) with about 5,000 question-answer pairs generated by Claude 3 Opus and validated by five annotators. For GDPR, Mixtral scored 0.71, 0.65, 0.92 and 0.88 and Command R+ 0.81, 0.78, 0.94 and 0.91. The advanced pipeline improved answer relevancy on GDPR from 0.75 to 0.92 over a baseline RAG.
- **Context length (Fig. 7):** precision and recall rise with the context window; both models stay under 50% at 4K tokens, while Command R+ exceeds 80% at 128K.
- **Crowdsourced study:** 270 students and 20 domain experts rated outputs for 28 GDPR and 20 Data Act use cases; experts rated 61% (GDPR) and 75% (Data Act) of outputs as correct.
- **Case studies (Table 6):** 20 projects; flagged-requirement counts per agent before and after RA-driven re-evaluation. Iteration corrected some errors but some false positives remained.
- **Monolithic baseline (Table 7):** a single Gemini 2.5 Pro prompt with the same inputs. The authors argue that architecture matters more than raw model capability.

The paper does not report precision, recall or F1 for compliance detection itself, because no ground-truth labels of non-compliant requirements exist. It reports counts and qualitative assessments instead. We follow the same approach and state this limitation explicitly in Section 11.

3. System architecture of our implementation

3.1 End-to-end flow

Figure 1 shows the system as built. A regulation and an SRS are each split into parent sections and chunked into child statements, which are embedded into two separate vector stores. Two checker agents on models from different vendors each produce a structured report. The report analyzer compares them, judges each disagreement and sends targeted revision requests back to the checkers; the revised reports are compiled into a final report that a human reviews. Review decisions are stored and recalled in later runs.



Every LLM call is cached on disk and written to trace.json (transparent retrieval / XAI).

Figure 1. Architecture of regcheck. Letters A to D follow the paper's component names.

3.2 Repository layout

Path	Lines	Responsibility
regcheck/config.py	59	Typed view of config.toml, the single place for models and knobs
regcheck/schemas.py	124	Pydantic models for all data crossing module boundaries or returned by an LLM
regcheck/llm.py	120	The only module that calls LLMs: retries, disk cache, trace, JSON extraction and repair
regcheck/prompts.py	200	Every prompt template, in the paper's persona plus Alpaca layout
regcheck/ingest.py	130	Document Pre-processor: load, split into sections, agentic or fixed chunking, index
regcheck/store.py	127	Vector store and retriever: FAISS, MMR, cross-encoder re-ranking, parents
regcheck/rag.py	50	CoT query decomposition and question answering
regcheck/checker.py	117	Compliance Checker agent: plan, screen, assess
regcheck/analyzer.py	172	Report Analyzer agent: diff, validate, optional web search, compile
regcheck/pipeline.py	106	Agent communication loop and the monolithic baseline
regcheck/memory.py	35	Expert-feedback memory: remember and recall
regcheck/__main__.py	63	Command-line interface: index, check, ask
app.py	156	Streamlit human-in-the-loop review application
eval/gen_qa.py, ragas_eval.py, tables.py	246	Ground-truth QA generation, RAGAs scoring, Tables 4 to 7
tests/test_core.py	289	Offline test suite (14 tests)
PLAN.md	544 (text)	Design specification, build log and record of departures from the paper

Table 3. Main files of the implementation.

3.3 Data model

Data that crosses a module boundary or comes back from an LLM is a Pydantic model. The main ones are:

- **Section**: a parent document, one regulation article (e.g. `gdpr:art32`) or one SRS section.
- **Chunk**: a child statement belonging to a Section; the unit that is embedded. For an SRS, one chunk is one atomic requirement.
- **Finding**: one checker's verdict on one requirement: cited clauses, rationale, status (compliant, partial or non_compliant), suggested rewrite and retrieved evidence. The rationale field comes before the status so that the model reasons before it decides.
- **Report**: one checker's findings plus its CoT plan.
- **Dispute**: a requirement flagged by exactly one checker, with the RA's judgement and revision message.
- **Issue** and **FinalReport**: the RA's final decision per flagged requirement, with confidence, both checkers' positions and evidence, plus a summary.
- **Feedback**: one human review decision with its justification and timestamp.

A requirement counts as *flagged* when its status is partial or non_compliant.

4. Detailed design of each component

4.1 LLM access layer (llm.py)

All model calls pass through two functions. `complete()` sends a chat request through *litellm*, which gives one call signature for every vendor, so each agent's model is a single configuration string. It retries rate-limit, server-overload, connection and timeout errors with exponential backoff (2 to 64 seconds), fails fast on daily-quota errors that will not clear, treats an empty reply as retryable (some Gemini models spend their whole budget on internal reasoning), caches every reply on disk under a hash of model, temperature and messages, and appends every prompt and response to a trace.

`complete_json()` asks for JSON in the prompt rather than relying on vendor-specific JSON modes, which differ between the three providers. It extracts the first JSON value in the reply that fits the target schema, tolerating code fences, surrounding prose and raw newlines inside strings, validates it with Pydantic and an optional extra check, and on failure retries once with the error message appended. A reply that fails validation is removed from the cache so that it is never replayed.

4.2 Document Pre-processor (ingest.py)

Section splitting

Splitting into parent sections is deterministic. For a regulation, the text is split at lines reading "Article n", accepting only the next expected number so that recitals and stray references are skipped. The following line becomes the article title, and EUR-Lex page headers and footnotes are stripped. GDPR yields its 99 articles. For an SRS, the text is split at shallow numbered headings ("3" or "3.1"); lines containing tabs (table-of-contents entries) or modal verbs such as *shall* (numbered requirements) are not headings. Front matter before the first heading is skipped, because the chunker turned table-of-contents entries into invented requirements. If fewer than three headings are found, fixed windows are used instead.

Agentic chunking

Each section is sent to the chunker agent, which returns a JSON list of self-contained statements. The regulation prompt implements the paper's rules: one obligation, right, condition or definition per statement; pronouns resolved and the actor named; exception clauses kept with their rule; cross-references kept explicit but never invented; no content added or dropped. The SRS prompt asks for one atomic requirement per statement in the original wording, fixes words broken by PDF extraction,

skips headings and glossaries, and returns an empty list when a section holds no requirement. Long sections are read in windows of 8,000 characters but remain one parent. A *fixed* mode with overlapping windows exists only for the baseline RAG. Indexing saves after every section and skips sections already indexed, so an interrupted run resumes without repeating paid calls.

4.3 Vector store and retriever (store.py)

Each document has its own directory holding `chunks.jsonl`, `sections.jsonl` and a FAISS `IndexFlatL2`, which gives the paper's separate policy and requirements stores. Search follows the paper's order:

- Embed the query with the BGE query instruction prefix; all vectors are L2-normalised, so squared L2 distance converts exactly to cosine similarity.
- Retrieve the top k = 15 chunks (exact search).
- Apply MMR to keep 10 candidates, balancing relevance against redundancy (lambda = 0.5). Candidate vectors are reconstructed from the index rather than stored twice.
- Re-rank with the cross-encoder and keep the top 5, most relevant first, so the most useful context sits at the start of the prompt.
- Map each chunk to its parent section and de-duplicate, so the full article text, not a fragment, goes to the LLM (parent-document retrieval).

Turning MMR, re-ranking and parents off gives plain top-k similarity, which is the baseline RAG used in Table 5. The embedding and re-ranking models are loaded once per process.

4.4 Chain-of-thought question answering (rag.py)

`decompose()` uses the paper's Fig. 3 prompt verbatim to break a question into three or four self-contained search queries. `answer()` answers each sub-query with its own retrieval, keeping the earlier sub-answers in the conversation, and then writes a final answer with citations. This is used by the `ask` command and by the RAGAs evaluation; the checker uses only the decomposition step.

4.5 Compliance Checker agent (checker.py)

A check by one agent follows the paper's four sub-prompts:

- **Plan.** The paper's high-level prompt ("As a GDPR compliance expert, analyze the software requirements specification provided regarding personal data handling...") is decomposed into sub-queries. Each regulation has a *focus* phrase in the configuration, so no keyword list is hard-coded.
- **Select (sub-prompt 1).** For each sub-query the top-k requirements are retrieved, and one screening call keeps those that concern the regulation, keeping a requirement when in doubt.
- **Map, evaluate and suggest (sub-prompts 2 to 4).** Requirements are assessed in batches of five. For each requirement the policy store is searched with the requirement text. The prompt contains the batch's articles once, each requirement with pointers to its most relevant articles, the regulation's reviewer hints and any recalled expert decisions. The model maps, evaluates and rewrites step by step.

Three safeguards were added after observing real model behaviour:

- **General obligations.** A requirement's own words only retrieve what it mentions: "encryption" finds Article 32 but never the storage-limitation principle. The checker therefore also supplies the top two articles for each plan sub-query, and any articles named in the hints, so the model can spot what a

requirement leaves out.

- **Clause guard.** Cited article identifiers are checked against those actually supplied in the prompt. A sub-clause such as `gdpr:art32.1(a)` is mapped to its parent `gdpr:art32` (with care that art3 never matches art32); invented ones are dropped and logged. This directly addresses the paper's concern about hallucinated article numbers.
- **One finding per requirement.** The reply must contain exactly one finding per requirement in the batch, or it is repaired or rejected.

The same assessment function is reused in the refinement round: the RA's revision note is appended to the retrieval query, which steers re-ranking towards the RA's criteria, and is shown in the prompt as a review note. There is therefore a single assessment code path.

4.6 Report Analyzer agent (analyzer.py)

- **diff** (prompts i and ii): because the reports are structured, the requirements flagged by exactly one checker are found by a set difference, with no LLM call.
- **validate** (prompts iii and iv): for each dispute, the RA sees the requirement, the flagging agent's rationale and the text of the articles it cited. It decides whether the rationale holds and writes a revision request. If the rationale is valid, the agent that missed the issue must re-evaluate; if not, the agent that flagged it must.
- **compile** (prompts v and vi): for every requirement flagged by at least one checker after refinement, the RA sees both final positions, the dispute history and recalled expert feedback, and returns the final status, clauses, rationale, a merged rewrite and a confidence score: 0.8 to 1 when both agreed in iteration 1, 0.5 to 0.8 when they agreed only after revision, below 0.5 when they still disagree. Requirements judged compliant are left out. A short summary is written by one further call.
- **Batching:** validation and compilation run in batches of five, because one reply for a real SRS (E-Store has over 20 issues) exceeded free-tier output limits.

Optional web search (paper Section 3.3.1)

When enabled, the RA first rewrites each dispute into a generic query (for example "GDPR retention limits for biometric data"). A lexical leak check blocks any query that copies four consecutive words of the requirement or contains the project name; a blocked query is never sent. Allowed queries go to DuckDuckGo through the *ddgs* package, and the top three snippets are added to the validation prompt with an instruction that the articles take precedence. Search errors skip the snippets rather than stop the check. The feature is off by default because it is the only one that sends data to a third party.

4.7 Agent communication loop (pipeline.py)

```
iteration 1:  cc1.check(SRS, regulation)   cc2.check(SRS, regulation)     (independent)
iteration 2:  disputes = diff(report1, report2)
              if no disputes: stop
              disputes = ra.validate(disputes)
              each agent re-assesses the requirements it must reconsider
final:        ra.compile(reports, disputes) -> final report with confidence
output:       data/runs/<time>-<srs>-<reg>/
              run.json  rounds.json  final.json  trace.json
```

The number of refinement rounds is configurable (one by default; the paper reports two iterations). A failed run still saves its trace for diagnosis. A full run on a real SRS takes roughly 25 LLM calls. The monolithic baseline (paper Section 6.4) sends the whole SRS and the whole regulation in one prompt to one long-context model and applies the same clause guard.

4.8 Human-in-the-loop review and memory (app.py, memory.py)

The Streamlit application lets a reviewer choose a regulation and an indexed SRS (or upload and index a new one), run a check, or open a past run. The report view shows summary counts, a sortable issue table and an expandable panel per issue with:

- the requirement, status, confidence and an agreement marker;
- the violated articles with their full text in tabs (regulation mapping);
- the RA's rationale and the two checkers' positions side by side (rationale visualisation);
- the retrieved evidence chunks (retrieval path) and the suggested rewrite;
- an Accept / Reject / Escalate control with a required justification.

Saving a decision writes `decisions.json` in the run directory and adds the decision to the feedback memory, a vector store whose entries are past requirements with the decision attached. When a later check assesses a requirement, decisions on similar requirements for the same regulation (cosine similarity at least 0.8) are recalled and injected into both the checker and RA prompts as past expert decisions. The threshold was measured on bge-large: a lightly reworded requirement scores about 0.9, while a different requirement on the same topic scores about 0.72. A Trace tab shows every prompt, response, recall, web query and dropped clause in order, and accepted rewrites and escalated items can be exported as Markdown.

The second feedback channel is the *hints* list in the configuration, where a reviewer adds a tuned sub-prompt after spotting a systematic miss. The GDPR configuration holds the paper's own Article 32 encryption hint and a second hint on retention periods and data-subject rights (Articles 5(1)(e) and 15 to 17), added after we observed that the checkers missed those gaps.

4.9 Explainability

Paper feature	How it is realised
Retrieval-path explanation	Each finding and issue stores the identifiers of the retrieved policy chunks; the app shows them with their text.
Rationale generation	Step-by-step rationale citing article identifiers, required before the status; both checkers' rationales are shown side by side.
Transparent retrieval process	trace.json records every prompt, response, model, cache hit, recall, web query and dropped clause; the app's Trace tab displays it.

Table 4. Explainability features.

4.10 Prompt design

Every prompt follows the shared rules drawn from the paper: a persona line, an Alpaca-style layout with Instruction, Input and Response headers, an instruction to rely only on the supplied context, and document text wrapped in tags such as <requirements> and <articles>, with a note that tagged text is data and not instructions (a defence against prompt injection from document content). Twelve templates cover chunking (policy and SRS), decomposition, the high-level task, screening, assessment, RA validation, RA compilation, summary, query sanitisation, ground-truth QA generation and the monolithic baseline. Output examples in the prompts show plain-string values, because one model returned nested objects without them.

5. Technology stack and configuration

Need	Choice	Reason
Language	Python 3.12 managed with uv	ML wheels are most reliable on 3.12; uv gives a locked, reproducible environment.
LLM access	litellm	One call signature for every vendor; a model is a configuration string.
Embeddings	sentence-transformers, bge-large-en-v1.5	As in the paper.
Re-ranker	CrossEncoder, mxbai-rerank-large-v1	As in the paper.
Vector index	faiss-cpu, IndexFlatL2	As in the paper; exact search suits a few thousand vectors.
Schemas	Pydantic v2	Validation of every LLM reply and module boundary.
Configuration	config.toml read with tomllib	Standard library; no extra dependency.
PDF text	pypdf	Reads regulation and SRS PDFs.
Review UI	Streamlit	A complete review interface in one file.
Evaluation	ragas 0.4.3 (optional group)	The paper's metric framework, kept out of the core install.
Web search	ddgs (optional group)	No API key required.
Quality	pytest, ruff	Offline tests and linting.

Table 5. Technology stack.

Agent	Model (free tier)	Temp.	Role
cc1	mistral/ministral-14b-latest	0.2	Compliance Checker 1; high-volume work
cc2	gemini/gemini-3.5-flash-lite	0.2	Compliance Checker 2
ra	cohere_chat/command-a-03-2025	0.1	Report Analyzer (few calls per run)
chunker	mistral/ministral-14b-latest	0.0	Agentic chunking
judge	gemini/gemini-3.1-flash-lite	0.0	QA generation and RAGAs judging
monolithic	gemini/gemini-3.6-flash	0.1	Single-prompt baseline (long context)

Table 6. Agent configuration. The three main agents use three different vendors.

Retrieval settings are k = 15, MMR keep = 10, lambda = 0.5, top-n = 5 after re-ranking and a maximum chunk size of 400 tokens. Pipeline settings are a batch size of five requirements per call, one refinement round and web search off. Each regulation block holds its name, file, focus phrase and reviewer hints; adding a new regulation is one configuration block and one index command, which supports the paper's adaptability claim (RQ4).

6. Design decisions and departures from the paper

6.1 Guiding principles

- **No agent or RAG frameworks.** We did not use LangChain, LlamaIndex, LangGraph, CrewAI or AutoGen. An agent is a configuration entry plus plain functions that build a prompt and call the LLM layer; orchestration is a loop. This keeps every step visible and testable, and avoids framework version churn.
- **One home per concern.** Only llm.py calls models, only prompts.py holds prompt text and only store.py embeds and retrieves.
- **No extra infrastructure.** No database servers, containers, web API or asynchronous code; everything persists as files under data/.
- **No single-implementation abstractions.** Swapping a model, regulation or retrieval setting is a configuration edit, not a new class.

- **Marked shortcuts.** Each deliberate simplification carries a comment that names its limit and the upgrade path.
- **Size budget.** About 1,900 lines for the whole project; the result is about 1,700 lines of application code.

6.2 Departures from the paper

Paper	Our implementation	Reason
Mixtral 8x7B, Command R+, Claude 3 Opus	Free-tier models from Mistral, Google and Cohere	Budget. The paper calls its model choice illustrative and the framework model-agnostic. Three vendors preserve the point that checkers should not share one model's biases.
RA compares free-text reports with prompts (i) and (ii)	Set difference over structured findings	Exact, free and testable. The RA's judgement (iii to vi) is still an LLM call.
Redis, RedisGraph and Neo4j memory	FAISS-backed feedback store in files, plus config hints	Same behaviour for a single-user project without three servers; RedisGraph is also discontinued.
Maximum chunk size 2,000 tokens	400 tokens per embedded chunk; full parent article sent to the LLM	bge-large truncates input at 512 tokens, so longer chunks would be embedded only in part.
Recitals included in the GDPR index	Articles only	Findings cite articles; recitals can be added if interpretive context is lacking.
About 5,000 human-validated QA pairs	About 40 generated pairs per document, 10 scored	Free-tier quotas and team size.
270 students and 20 experts	The team's own accept, reject and escalate decisions	Scale.
Context-length study (Fig. 7)	Not reproduced	Requires paid long-context runs.
Web search with query sanitisation	Implemented, off by default, with a leak check	Least critical component and the only one sending data to a third party.
Two refinement iterations	One refinement round by default (configurable)	Quota; most disputes settled in one round.

Table 7. Deliberate departures from the paper.

6.3 Other notable decisions

- **JSON by prompt, not by vendor mode**, with tolerant extraction and one repair retry, because the three providers' structured-output features differ.
- **Disk cache for every call**, so a crash or re-run on a free tier costs nothing, and evaluation can be repeated reproducibly.
- **Screening keeps requirements when in doubt**, trading some cost for fewer missed requirements.
- **Batches of five** requirements per call, balancing call count against output-length limits.
- **Clause identifiers from supplied context only**, so every cited article exists and was shown to the model.
- **Confidence bands tied to agreement history**, so the score has a defined meaning for reviewers.
- **Plain-text values with length limits** in checker prompts, after long markdown rationales broke JSON.

7. Implementation process and practical lessons

7.1 Phased plan

The work followed a phased plan written before coding, with an acceptance check at the end of each phase.

Phase	Content	Acceptance check	Status
0 Setup	Environment, configuration, keys, documents	Imports work, one smoke-test call per agent	Done
1 Plumbing	config, schemas, llm	Offline JSON repair test	Done
2 Pre-processor and retriever	ingest, store, index command	GDPR gives 99 articles; encryption query finds Article 32, erasure finds Article 17	Done
3 CoT QA	rag, ask command	Consent question cites Article 7	Done
4 Compliance Checker	checker, toy SRS	Plain-text passwords non-compliant (Art. 32); colour scheme not flagged	Done (4 of 6 toy requirements flagged by both agents)
5 Report Analyzer and pipeline	analyzer, pipeline, check command	Offline end-to-end dispute test; live run writes all files	Done
6 Human in the loop	memory, app	Reject, re-run, recalled feedback lowers or removes the issue	Built and unit-tested; live check not yet recorded
7 Evaluation	eval scripts	Tables 4 to 7 for two regulations and two SRS	Done
8 Web search (optional)	query sanitisation and search	Only abstracted queries leave the machine	Done

Table 8. Build phases.

7.2 Problems met and how they were solved

- **Free-tier model access.** The Mistral key reached only the small *ministral* models; some Gemini models were unstable (overload errors and hangs), and one allowed only 20 requests per day. Agents were moved between models, quotas were tracked per model, and daily-quota errors now fail fast.
- **Empty replies.** Gemini 3.x models can use their entire output budget on hidden reasoning. We never set a small maximum-token limit and treat empty text as a retryable error.
- **Broken JSON.** Models quoted lists of ids in prose before the real answer, put raw newlines and stray double quotes inside strings, and returned nested objects. Tolerant extraction, schema-first matching, explicit output examples and a ban on double quotes inside values fixed these.
- **Invented article numbers.** Ministral cited paragraph-level ids that were never supplied; the clause guard maps them to their parent or drops them.
- **Gap blindness.** Retrieval by a requirement's own words never surfaces obligations it omits. Plan sub-queries and hint-named articles were added as general obligations.
- **Data Act screening.** With a focus naming only connected products, both checkers discarded every requirement; the focus now lists all six subject-matter areas of Article 1.
- **Long timeouts.** Long RA replies on Cohere exceeded 120 seconds; the timeout is now 300 seconds.
- **Output limits.** One RA reply for 22 issues exceeded free-tier output caps, which led to batching.
- **Library drift.** ragas 0.4.3 imports a module removed in langchain-community 0.4, so the evaluation group pins the older version. ragas telemetry is switched off before import.

8. Testing

One offline test file covers the logic that can fail silently. It needs no network or models: the LLM call and the embedding function are replaced by scripted fakes (a hashed bag-of-words embedding). On 7 November 2026 the suite ran with 13 tests passing and 1 skipped (the RAGAs adapter test, which needs the optional evaluation packages), and the code passed ruff linting.

Test	What it guards
JSON repair	A fenced, malformed reply is repaired by the retry, and the retry prompt carries the error.
Validation hook	A custom check is applied and the call gives up after one repair.
JSON extraction (2 tests)	Prose and off-schema JSON are skipped; broken JSON is reported, not an inner fragment.
Retry and cache	Transient errors and empty text are retried; the second call is served from cache.
Regulation splitting	Articles kept, recitals, duplicate headings, page headers and footnotes dropped.
SRS splitting	Table-of-contents lines and numbered requirements are not treated as headings.
MMR	A diverse candidate is preferred to a near-duplicate.
Clause mapping	Sub-clauses map to their article; art3 never matches art33.
Pipeline end to end	Two agents disagree; a dispute is raised, only the agent that missed it is re-assessed, and the final report contains the issue with both positions.
Hinted articles	"Articles 15 to 17" expands to three articles; unknown ones are skipped.
Memory recall	Recall works for a similar requirement under the same regulation only.
RAGAs judge adapter	ragas prompts reach the LLM layer and its schemas parse our replies.
Web search privacy	Only the abstracted query is sent; a query naming the project is blocked.

Table 9. Offline test suite.

Live acceptance checks used a six-requirement toy SRS. Both checkers flagged the same four requirements, including plain-text password storage (non-compliant under Article 32), and neither flagged the colour scheme requirement.

9. Evaluation

9.1 Setup

- **Documents:** GDPR (Regulation (EU) 2016/679), the EU Data Act (Regulation (EU) 2023/2854), and two SRS documents from the PURE dataset: E-Store and Video Search. Video Search appears in all four of the paper's tables and is the smallest of them (77 requirement chunks).
- **Ground truth:** question-answer pairs generated with the paper's prompt by the judge model from ten sampled sections per document (about 40 pairs each), shuffled with a fixed seed. Questions that refer to "the document" or "section 2" were filtered out because no retriever can answer them (6 of 40 for E-Store, 4 of 42 for Video Search). The pairs have not yet been reviewed by hand.
- **RAGAs:** ragas 0.4.3 with our judge model through a small adapter, so judging is cached and traced; the pipeline's own bge-large model serves as the embedding model; answer relevancy uses strictness 1 to save calls. Ten questions per document and configuration. An advanced question costs about 25 judge calls.
- **Advanced vs baseline:** advanced = agentic chunks, CoT, MMR, re-ranking and parent articles; baseline = fixed-size chunks and a single plain top-n similarity search.
- **Case studies:** full multi-agent runs and monolithic runs for each SRS and regulation pair.

9.2 RAG quality (Table 4)

Document	Model	#Q	Context precision	Context recall	Answer relevancy	Faithfulness
Data Act	ministral-14b	10	0.845	1.000	0.852	0.842
E-Store	ministral-14b	10	0.750	0.800	0.955	0.865
GDPR	ministral-14b	10	0.798	1.000	0.808	0.737

Document	Model	#Q	Context precision	Context recall	Answer relevancy	Faithfulness
Video Search	ministral-14b	10	0.650	0.700	0.780	0.729
Data Act	gemini-3.5-flash-lite	10	0.800	1.000	0.899	1.000
E-Store	gemini-3.5-flash-lite	10	0.750	0.800	0.851	0.883
GDPR	gemini-3.5-flash-lite	10	0.812	1.000	0.907	0.959
Video Search	gemini-3.5-flash-lite	10	0.633	0.700	0.619	0.900

Table 10. Advanced RAG per model and document.

Both models share one retriever, so context recall is identical and context precision differs by at most 0.05. The models differ in how they answer. Gemini answers only from the context, with faithfulness of 0.88 to 1.0 against 0.73 to 0.87 for Ministral. When retrieval misses, Gemini says it cannot answer, and RAGAs scores such non-answers 0 for relevancy; three such answers explain its low 0.62 relevancy on Video Search. Ministral fills gaps with plausible text, which costs faithfulness. Our GDPR context recall (1.0) is higher than the paper's (0.65 to 0.78), but on ten questions this difference is not conclusive.

9.3 Baseline vs advanced RAG (Table 5)

Document	Size	CP base	CP adv	CR base	CR adv	AR base	AR adv	F base	F adv
Data Act	33K	0.687	0.845	0.900	1.000	0.822	0.852	0.884	0.842
E-Store	3K	0.833	0.750	1.000	0.800	0.978	0.955	0.871	0.865
GDPR	47K	0.822	0.798	0.900	1.000	0.782	0.808	0.798	0.737
Video Search	5K	0.842	0.650	0.900	0.700	0.782	0.780	0.775	0.729

Table 11. Baseline vs advanced RAG with ministral-14b, 10 questions each (CP context precision, CR context recall, AR answer relevancy, F faithfulness; size in estimated tokens).

The advanced pipeline helps on the long regulations: on the Data Act, context precision rises from 0.69 to 0.85 and recall from 0.90 to 1.00, and on GDPR recall and relevancy improve. On the short SRS documents it is worse: Video Search precision falls from 0.84 to 0.65 and recall from 0.90 to 0.70. Our reading is that on a document of a few thousand tokens, plain similarity over small fixed chunks already finds the right passage, while CoT decomposition and parent-section expansion bring in broader, partly irrelevant context and occasionally steer sub-queries away from the original question. Faithfulness falls slightly everywhere with Ministral, consistent with its tendency to elaborate when given more text. This differs from the paper, which reports improvement on most datasets, and is worth stating plainly: the extra machinery pays off when the corpus is long.

9.4 Case studies (Table 6)

SRS	Regulation	It. 1 cc1	It. 1 cc2	It. 1 common	It. 2 cc1	It. 2 cc2	It. 2 common
E-Store	GDPR	21	20	17	23	21	21
E-Store	GDPR + web search	21	20	17	21	21	21
Video Search	GDPR	5	3	2	5	4	4
E-Store	EU Data Act	0	0	0	no disputes		
Video Search	EU Data Act	0	0	0	no disputes		
Toy SRS	GDPR	4	4	4	no disputes		

Table 12. Flagged requirements per checker before and after RA-driven refinement.

Refinement increased agreement in every GDPR case: on E-Store common findings rose from 17 to 21 and on Video Search from 2 to 4. With web search enabled on E-Store, all seven generated queries were generic and none was blocked; four of the seven searches returned no results from the free endpoint. The RA's verdicts were unchanged, but cc1 withdrew all three of its rejected flags instead of one, so the final count fell from 23 to 21.

Neither checker flagged anything under the Data Act. Both vendors agree independently, and neither SRS involves connected products, data-sharing contracts or cloud switching. The paper reports 12 to 14 Data Act issues for Video Search. We report our zero rather than loosen the prompts to manufacture findings.

9.5 Framework vs monolithic baseline (Table 7)

SRS	Regulation	Framework (final issues)	Monolithic (gemini-3.6-flash)	Reviewer accepted
E-Store	GDPR	23	6	not reviewed
E-Store	GDPR + web search	21	6	not reviewed
Video Search	GDPR	5	4	not reviewed
E-Store	EU Data Act	0	1	not reviewed
Video Search	EU Data Act	0	3	not reviewed
Toy SRS	GDPR	4	not run	not reviewed

Table 13. Multi-agent framework compared with a single-prompt baseline.

On GDPR the framework finds far more issues than the monolithic model on E-Store (23 against 6) and slightly more on Video Search (5 against 4). Under the Data Act the monolithic model flags a few requirements the framework does not. We examined them: the three Video Search flags treat a favourites list as data of a "related service", but Article 2 defines a related service as one tied to a connected product, so they are overreach. The single E-Store flag is a genuine miss on our side: Article 3(2) gives sellers of connected products a pre-contract information duty, and E-Store sells configurable computers. Our screening step trades this kind of indirect applicability for fewer false positives.

A higher count is not automatically better. Without reviewer decisions (the last column) we cannot yet say how many of the 23 E-Store issues are correct; this is the most important open item (Section 13).

10. Merits

- **Complete coverage of the paper.** All four components, the six RA prompts, the iteration loop, explainability and the optional web search are implemented, and every departure is documented.
- **Independent cross-checking.** Checkers from different vendors disagree on real cases, and the RA dialogue measurably raises agreement.
- **Grounded citations.** The clause guard guarantees that every cited article exists and was shown to the model, addressing a key risk of LLM-based legal analysis.
- **Transparency.** Every prompt, reply, recall and dropped citation is recorded and viewable, which makes both debugging and auditing practical.
- **Human control.** No finding is final without a reviewer, and decisions feed back into later runs.
- **Low cost and reproducibility.** Free tiers only, a disk cache for every call, a locked environment and resumable indexing.
- **Simplicity.** About 1,700 lines with no agent framework or servers; one configuration file controls models, retrieval and regulations; a new regulation needs one block and one command.

- **Privacy awareness.** Web search sends only abstracted queries after a leak check, and the tool warns users not to send confidential documents to free-tier providers.
- **Testability.** The tricky logic is covered by offline tests that run in about 20 seconds.
- **Honest evaluation.** Results that contradict the paper (Table 5, Data Act counts) are reported and explained rather than tuned away.

11. Limitations and demerits

- **No ground truth for compliance.** As in the paper, there are no labels of truly non-compliant requirements, so precision, recall and F1 for detection cannot be computed; counts can reward over-flagging.
- **No human review yet.** None of the evaluation runs has reviewer decisions, so false positives are unmeasured, and the live acceptance check for the feedback loop (reject, re-run, see the effect) has not been recorded.
- **Small evaluation.** Two SRS documents against the paper's twenty projects, and ten questions per configuration against hundreds; differences of a few hundredths are within noise. The generated QA pairs are unreviewed, and the same model family generates and judges them.
- **Data Act results are empty.** Neither SRS falls within the Act's main scope, so that half of the evaluation shows little; the screening step also misses indirect applicability (Article 3(2)).
- **Weaker models than the paper.** Free-tier small models are more error-prone; quotas constrain how much can be run in a day, and model availability changes often.
- **Narrow dispute definition.** Only flag-versus-no-flag disagreements reach the RA. Disagreements on severity (partial vs non-compliant) or on which articles apply are not disputed.
- **Confidence is model-assigned** within prescribed bands, not computed, and is not calibrated.
- **Simplified memory.** One vector store replaces the paper's short- and long-term memories; recall is purely by similarity with a fixed 0.8 threshold, so heavy paraphrases are missed, and a changed decision leaves the old one recallable. Escalation is stored but triggers no workflow.
- **Lexical leak check.** The web-search privacy check matches copied words and the project name only; client or product names that differ from the file name could pass.
- **Deterministic SRS splitting** relies on numbered headings and may need adjustment for unusual document formats.
- **Context-length study not reproduced**, and recitals are not indexed.
- **Free-tier data use.** Providers may use prompts for training, which rules the defaults out for confidential SRS documents.
- **Heavy local install.** About 2 GB of models plus PyTorch; data, indexes and runs are not in the repository, so reproducing results requires API keys, the documents and re-indexing.

12. What we learnt

- **Retrieval decides the outcome.** A requirement's own words retrieve only what it mentions, while compliance gaps are about what it omits. Supplying general obligations was the single biggest improvement to the checkers.
- **Structure beats free text between agents.** Structured findings made disagreement detection exact and free, and kept the LLM for judgement, where it adds value.

- **LLM output must be treated as untrusted input.** Schema validation, repair retries and the clause guard were necessary, not optional.
- **More retrieval machinery is not always better.** MMR, re-ranking, CoT and parent documents helped on long regulations and hurt on short SRS documents.
- **Different models fail differently.** One model refused when unsure and another elaborated; using two vendors exposed both behaviours and justified the multi-agent design.
- **Free tiers shape engineering.** Quotas forced caching, batching, fail-fast quota handling and careful assignment of agents to models.
- **Evaluation of compliance tools is hard** without labelled data; human review is the only reliable measure of false positives, which explains the paper's reliance on crowdsourcing.
- **Honest reporting matters.** It was tempting to loosen prompts to match the paper's Data Act numbers; reporting zero with an explanation is the more defensible result.

13. Future work

13.1 Immediate next steps

- **Add one or two further SRS documents** from the PURE dataset (for example CCTNS, Virtual Education or E-Procurement), preferably one involving connected devices or cloud services so that the Data Act evaluation becomes meaningful. Each needs one index command, two check runs and one monolithic run.
- **Add and review QA tests:** hand-review the generated QA files, add question-answer pairs written by the team, and raise the number of scored questions where quotas allow.
- **Carry out the human review** of the E-Store and Video Search GDPR runs in the app, which fills the "Reviewer accepted" column and gives a false-positive measure, and record the live feedback-loop check.

13.2 Further improvements

- Build a small hand-labelled set of compliant and non-compliant requirements to compute precision, recall and F1, which the paper does not report.
- Extend disputes to severity and clause disagreements, and compute confidence from agreement history instead of asking the model.
- Add a screening hint for indirect applicability (for example Article 3(2) for shops selling devices).
- Add recitals as interpretive context and test whether retrieval improves.
- Replace the lexical leak check with named-entity detection.
- Run a second refinement round and a paid long-context run to reproduce the context-length study.

14. Conclusion

We studied the multi-agent RAG framework of Das et al. and implemented it in full as a compact, tested and transparent Python tool. Two checker agents from different vendors, a report analyzer that resolves disagreements through revision requests, and a human review step with feedback memory work together end to end on real SRS documents and real regulations, using only free-tier models. The evaluation confirms the paper's central claims in part: cross-checking raises agreement, the framework finds more GDPR issues than a single prompt, and advanced retrieval helps on long regulations. It also shows where the claims do not carry over to our setting, namely short SRS documents and Data Act applicability. The

main remaining work is human review of the findings and a broader evaluation with more SRS documents and reviewed QA tests, which is the next stage of the project.

15. References

[1] S. Das, N. Deb, N. Chaki and A. Cortesi. A Multi-Agent RAG Framework for Regulatory Compliance Checking of Software Requirements. *ACM Transactions on Software Engineering and Methodology*, 35(8), Article 223, 2026. doi:10.1145/3785472.

[2] Regulation (EU) 2016/679 of the European Parliament and of the Council (General Data Protection Regulation). Official Journal of the European Union, L 119, 2016.

[3] Regulation (EU) 2023/2854 of the European Parliament and of the Council (Data Act). Official Journal of the European Union, 2023.

[4] A. Ferrari, G. O. Spagnolo and S. Gnesi. PURE: A Dataset of Public Requirements Documents. *IEEE International Requirements Engineering Conference (RE)*, 2017.

[5] S. Es, J. James, L. Espinosa-Anke and S. Schockaert. RAGAS: Automated Evaluation of Retrieval Augmented Generation. *EACL System Demonstrations*, 2024.

[6] S. Xiao, Z. Liu, P. Zhang and N. Muennighoff. C-Pack: Packaged Resources to Advance General Chinese Embedding (BGE models). 2023.

[7] J. Carbonell and J. Goldstein. The Use of MMR, Diversity-Based Reranking for Reordering Documents and Producing Summaries. *SIGIR*, 1998.

[8] N. F. Liu et al. Lost in the Middle: How Language Models Use Long Contexts. *Transactions of the ACL*, 2024.

[9] J. Johnson, M. Douze and H. Jégou. Billion-scale similarity search with GPUs (FAISS). *IEEE Transactions on Big Data*, 2019.

16. Appendix: running the system

```
uv sync --group dev --group eval --group web
# .env: MISTRAL_API_KEY, GEMINI_API_KEY, COHERE_API_KEY

uv run python -m regcheck index gdpr data_act
uv run python -m regcheck index data/srs/e-store.txt
uv run python -m regcheck ask "When is consent required?" --doc gdpr

uv run python -m regcheck check e-store --reg gdpr
uv run python -m regcheck check e-store --reg gdpr --monolithic
uv run streamlit run app.py

uv run python -m eval.gen_qa gdpr --n 10
uv run python -m regcheck index gdpr --fixed
uv run python -m eval.ragas_eval gdpr --agent cc1 --mode advanced --limit 10
uv run python -m eval.ragas_eval gdpr --agent cc1 --mode baseline --limit 10
uv run python -m eval.tables

uv run pytest && uv run ruff check .
```

Adding a new SRS document requires placing the file under data/srs/, running the index command, then the check and monolithic commands for each regulation, and finally regenerating the tables. Adding a new regulation requires one [regulations.x] block in config.toml with its name, file, focus and hints, and one index command.